SLIP 4 -

Q1

```
import matplotlib.pyplot as plt

payment_methods = ["UPI", "Credit Card", "Debit Card", "Cash", "Net Banking"]
customers = [150, 80, 60, 40, 30]

plt.pie(customers, labels=payment_methods, autopct="%1.1f%%")
plt.title("Preferred Payment Methods")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

df = pd.read_csv("online_gaming_behavior_dataset.csv").dropna()

for col in df.select_dtypes(include="object").columns:
    if col != "Player_Status":
        df[col] = LabelEncoder().fit_transform(df[col])

target_candidates = ["Player_Status", "player_status", "Status", "Category",
"Player_Type"]
target = next((c for c in target_candidates if c in df.columns), df.columns[-1])

y = LabelEncoder().fit_transform(df[target])
X = df.drop(columns=[target])
X = X.select_dtypes(include="number")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Or Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
df = pd.read

_
csv("Ecommerce Customers.csv")
df = df.select

_
dtypes(include=
"number").dropna()
X = StandardScaler().fit

_
transform(df)
kmeans = KMeans(n

_
clusters=3, random

_
```

```
state=42, n
_
init=10)
labels = kmeans.fit
_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap=
"viridis")
plt.xlabel(df.columns[0])
plt.ylabel(df.columns[1])
plt.title("E-Commerce Customer Clusters")
plt.show()
```

SLIP 5 -

Q1

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.DataFrame({
    "Customer_ID": [1, 2, 3, 4, 5],
    "Gender": ["Male", "Female", "Female", "Male", "Female"],
    "City": ["Nashik", "Pune", "Mumbai", "Nashik", "Pune"],
    "Membership_Type": ["Gold", "Silver", "Gold", "Bronze", "Silver"]
})

print(df)
le = LabelEncoder()

for col in ["Gender", "City", "Membership_Type"]:
    df[col] = le.fit_transform(df[col])

print(df)
```

Q2

```
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

netflix = {"A", "B", "C", "D", "E", "F"}
amazon = {"D", "E", "F", "G", "H"}

only_netflix = netflix - amazon
only_amazon = amazon - netflix
both = netflix & amazon

venn2([netflix, amazon], set_labels=("Netflix", "Amazon Prime"))
plt.title("Netflix and Amazon Prime Users")
plt.show()

values = [len(only_netflix), len(only_amazon), len(both)]
labels = ["Only Netflix", "Only Amazon Prime", "Both"]

plt.pie(values, labels=labels, autopct="%1.1f%%")
plt.title("Streaming Service Users")
plt.show()
```

Or Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score,
recall_score, f1_score

df = pd.read_csv("spambase.csv").dropna()

target = "class" if "class" in df.columns else df.columns[-1]
X = df.drop(columns=[target])
y = df[target]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
```

SLIP 6 -

Q1

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Car_Model": ["A", "B", "C", "D", "E"],
    "Engine_Size": [1.2, 1.5, 1.8, 2.0, 2.2],
    "Fuel_Efficiency": [22, 20, 18, 16, 14]
})

plt.scatter(df["Engine_Size"], df["Fuel_Efficiency"],
            c=range(len(df)), cmap="viridis")
```

```
plt.xlabel("Engine Size (litres)")
plt.ylabel("Fuel Efficiency (km/L)")
plt.title("Engine Size vs Fuel Efficiency")
plt.show()
```

Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("collegiate_athlete_injury_dataset.csv").dropna()
xcol = "Training Hours Per Week"
ycol = "Performance Score"

X = df[[xcol]]
y = df[ycol]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("R2 Score:", r2_score(y_test, pred))

plt.scatter(X_test, y_test)
plt.plot(X_test, pred)
plt.xlabel(xcol)
plt.ylabel(ycol)
plt.title("Training Hours vs Performance")
plt.show()
```

Or Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("Mall_Customers.csv").dropna()

for col in df.select_dtypes(include="object").columns:
    df[col] = LabelEncoder().fit_transform(df[col])

target = df.columns[-1]
X = df.drop(columns=[target])
y = df[target]

X = X.select_dtypes(include="number")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

SLIP 7-

Q1

```
import matplotlib.pyplot as plt
courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]
students = [60, 45, 75, 50, 35]

plt.bar(courses, students, color=["red", "blue", "green", "orange", "purple"])
plt.xlabel("Courses")
plt.ylabel("Number of Students")
plt.title("Students Enrolled in Courses")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("student_dataset_v2.csv").dropna()

for col in df.select_dtypes(include="object").columns:
    if col.lower() not in ["performance", "category", "grade", "result"]:
        df[col] = LabelEncoder().fit_transform(df[col])

target = next(
    (c for c in ["Performance", "Performance_Category", "Category", "Grade",
"Result"]
     if c in df.columns),
    df.columns[-1]
)

y = LabelEncoder().fit_transform(df[target])
```

```
X = df.drop(columns=[target]).select_dtypes(include="number")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Or Q2

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Basmati Rice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],
    ["Atta", "Sugar", "Tea", "Milk"],
    ["Rice", "Moong Dal", "Salt", "Cooking Oil"],
    ["Wheat Flour", "Chickpeas", "Turmeric Powder", "Cumin Seeds"],
    ["Poha", "Peanuts", "Onions", "Cooking Oil"],
    ["Basmati Rice", "Paneer", "Garam Masala", "Tomatoes"],
    ["Atta", "Potatoes", "Onions", "Cooking Oil"]
]

te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)
```

```
df = pd.DataFrame(encoded, columns=te.columns_)

for support in [0.3, 0.4]:
    freq = apriori(df, min_support=support, use_colnames=True)
    print(freq)
    print(association_rules(freq, metric="confidence", min_threshold=0.5))
```

SLIP 11 -

Q1

```
import matplotlib.pyplot as plt

departments = ["HR", "Finance", "IT", "Sales", "Marketing"]
employees = [25, 30, 60, 45, 35]

plt.bar(departments, employees)
plt.xlabel("Departments")
plt.ylabel("Number of Employees")
plt.title("Employees by Department")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

df = pd.read_csv("train.csv").dropna()

target = next(
    (c for c in ["price_range", "Price_Range", "price_category"] if c in df.columns),
```

```
    df.columns[-1]
)

X = df.drop(columns=[target]).select_dtypes(include="number")
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Or Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("Ecommerce Customers.csv")
df = df.select_dtypes(include="number").dropna()

X = StandardScaler().fit_transform(df)

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap="viridis")
plt.scatter(
    model.cluster_centers_[:, 0],
    model.cluster_centers_[:, 1],
    marker="X", s=200
)

plt.xlabel(df.columns[0])
plt.ylabel(df.columns[1])
plt.title("E-Commerce Customer Clusters")
plt.show()
```

SLIP 14 -

Q1

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

df = pd.DataFrame({
    "Price": [500, 700, 450, 900, 650],
    "Rating": [4.2, 4.5, 3.9, 4.8, 4.3],
    "Number_of_Sales": [120, 180, 90, 250, 160]
})

Z = linkage(df, method="ward")
dendrogram(Z)

plt.title("Product Similarity Dendrogram")
plt.xlabel("Products")
plt.ylabel("Distance")
plt.show()
```

Q2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("car data.csv").dropna()

X = df[["Vehicle Age", "Present Price", "Kms Driven"]]
y = df["Selling Price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

pred = model.predict(X_test_poly)

print("R2 Score:", r2_score(y_test, pred))

x_plot = X_test.iloc[:, 0].values
order = np.argsort(x_plot)

plt.scatter(x_plot, y_test)
plt.plot(x_plot[order], pred[order])
plt.xlabel("Vehicle Age")
plt.ylabel("Selling Price")
```

```
plt.title("Polynomial Regression")
plt.show()
```

Or Q2

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
    "Brand": ["Apple", "Samsung", "OnePlus", "Vivo"],
    "Model": ["A1", "S1", "O1", "V1"],
    "Price": [70000, 50000, 35000, 25000],
    "RAM": [8, 8, 12, 8]
})

df2 = pd.DataFrame({
    "Model": ["A1", "S1", "O1", "V1"],
    "Storage": [128, 256, 256, 128],
    "Battery_Capacity": [4000, 5000, 4500, 5000],
    "Operating_System": ["iOS", "Android", "Android", "Android"]
})

merged = pd.merge(df1, df2, on="Model")

print(df1)
print(df2)
print(merged)

le = LabelEncoder()
merged["Brand_Encoded"] = le.fit_transform(merged["Brand"])

merged = pd.get_dummies(
    merged, columns=["Operating_System"], dtype=int
)

print(merged)
```

SLIP 15 -

Q1

```
import pandas as pd

df = pd.DataFrame({
    "Person_ID": [1, 2, 3, 4, 5, 6, 7, 8],
    "Age": [18, 22, 27, 31, 36, 42, 50, 60]
})

df["Age_Bin"] = pd.qcut(
    df["Age"], q=4,
    labels=["Q1", "Q2", "Q3", "Q4"]
)

print(df)
```

Q2

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify
from mpl_toolkits.mplot3d import Axes3D

df = pd.DataFrame({
    "City": ["Nashik", "Pune", "Mumbai", "Nagpur", "Nashik"],
    "Area": [267, 331, 603, 228, 250],
    "Average Temperature": [25, 27, 28, 29, 26],
    "Rainfall": [700, 750, 2200, 1100, 800],
    "Population Density": [6000, 7000, 20000, 5000, 5500]
})

plt.figure(figsize=(8, 5))
squarify.plot(sizes=df["Population Density"], label=df["City"], alpha=0.8)
plt.axis("off")
```

```
plt.title("Population Density Treemap")
plt.show()

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

ax.scatter(df["Area"], df["Average Temperature"], df["Rainfall"])

ax.set_xlabel("Area")
ax.set_ylabel("Average Temperature")
ax.set_zlabel("Rainfall")
ax.set_title("3D Scatter Plot")

plt.show()
```

Or Q2

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Basmati Rice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],
    ["Atta", "Sugar", "Tea", "Milk"],
    ["Rice", "Moong Dal", "Salt", "Cooking Oil"],
    ["Wheat Flour", "Chickpeas", "Turmeric Powder", "Cumin Seeds"],
    ["Poha", "Peanuts", "Onions", "Cooking Oil"],
    ["Basmati Rice", "Paneer", "Garam Masala", "Tomatoes"],
    ["Atta", "Potatoes", "Onions", "Cooking Oil"]
]

te = TransactionEncoder()
df = pd.DataFrame(
    te.fit_transform(transactions),
    columns=te.columns_
)
```

```
for support in [0.3, 0.4]:
    freq = apriori(df, min_support=support, use_colnames=True)
    print(freq)
    print(association_rules(freq, metric="confidence", min_threshold=0.5))
```

SLIP 16 -

Q1

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Study Hours": [2, 3, 4, 5, 6, 7],
    "Attendance": [70, 75, 80, 85, 90, 95],
    "Assignment Score": [55, 60, 65, 75, 82, 90],
    "Exam Score": [50, 58, 64, 72, 84, 92]
})

corr = df.corr()
print(corr)

sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heat Map")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df = pd.DataFrame({
    "Temperature": [20, 25, 30, 35, 40],
    "Humidity": [40, 50, 60, 70, 80],
```

```
    "Wind-Speed": [5, 8, 10, 12, 15],
    "Pressure": [1000, 1005, 1010, 1015, 1020],
    "Rainfall": [10, 20, 30, 40, 50]
})

cols = df.columns

minmax = MinMaxScaler()
standard = StandardScaler()
normalizer = Normalizer()

df_minmax = pd.DataFrame(minmax.fit_transform(df), columns=cols)
df_standard = pd.DataFrame(standard.fit_transform(df), columns=cols)
df_normalized = pd.DataFrame(normalizer.fit_transform(df), columns=cols)

print(df_minmax)
print(df_standard)
print(df_normalized)
```

Or Q2

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Library", "Seminar"],
    ["Workshop", "Project"],
    ["Library", "Assignment"],
    ["Seminar", "Workshop"],
    ["Assignment", "Seminar"],
    ["Workshop", "Assignment"],
    ["Project", "Library"],
    ["Library", "Seminar", "Project"],
    ["Library", "Seminar", "Workshop"]
]
```

```
te = TransactionEncoder()
df = pd.DataFrame(
    te.fit_transform(transactions),
    columns=te.columns_
)

for support in [0.2, 0.3]:
    freq = apriori(df, min_support=support, use_colnames=True)
    print(freq)
    print(association_rules(freq, metric="confidence", min_threshold=0.5))
```

SLIP 17 -

Q1

```
import pandas as pd

student_details = pd.DataFrame({
    "Student_ID": [1, 2, 3, 4, 5],
    "Name": ["A", "B", "C", "D", "E"],
    "Gender": ["M", "F", "M", "F", "M"]
})

academic_records = pd.DataFrame({
    "Student_ID": [1, 2, 3, 4, 5],
    "Marks": [80, 75, 90, 68, 85],
    "Attendance": [90, 85, 95, 75, 88]
})

integrated = pd.merge(
    student_details, academic_records, on="Student_ID"
)

print(student_details)
print(academic_records)
print(integrated)
```

Q2

```
import pandas as pd
import matplotlib.pyplot as plt
import squarify

df = pd.DataFrame({
    "City": ["Nashik", "Pune", "Mumbai", "Nagpur", "Aurangabad"],
    "Population": [20, 50, 125, 30, 15],
    "GDP": [50, 120, 400, 80, 40],
    "Literacy Rate": [85, 90, 92, 88, 82]
})

plt.figure(figsize=(8, 5))
squarify.plot(sizes=df["Population"], label=df["City"])
plt.axis("off")
plt.title("City-wise Population Treemap")
plt.show()

fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")

ax.scatter(df["Population"], df["GDP"], df["Literacy Rate"])

ax.set_xlabel("Population")
ax.set_ylabel("GDP")
ax.set_zlabel("Literacy Rate")

plt.show()
```

Or Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

df = pd.read_csv("Students_Grading_Dataset.csv").dropna()

for col in df.select_dtypes(include="object").columns:
    df[col] = LabelEncoder().fit_transform(df[col])

target = next(
    (c for c in ["Performance", "Performance_Category", "Category", "Grade",
"Result"]
     if c in df.columns),
    df.columns[-1]
)

X = df.drop(columns=[target]).select_dtypes(include="number")
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

SLIP 18 -

Q1

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    "Emp_ID": [1, 2, 3, 3, 4, 5],
    "Age": [25, 30, np.nan, np.nan, 45, 100],
    "Dept": ["HR", "IT", "Sales", "Sales", "Finance", "IT"],
    "Years_Experience": [2, 5, 8, 8, 15, 20],
    "Performance_Rating": [3, 4, 5, 5, 4, 5]
})

df["Age"] = df["Age"].fillna(df["Age"].median())
df["Dept"] = df["Dept"].fillna(df["Dept"].mode()[0])

df = df.drop_duplicates()

for col in ["Age", "Years_Experience", "Performance_Rating"]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower) & (df[col] <= upper)]
print(df)
```

Q2

```
import pandas as pd
import plotly.express as px
import squarify
```

```
import matplotlib.pyplot as plt

df = pd.read_csv("worldcities.csv").dropna(
    subset=["lat", "lng", "population"]
)

fig = px.scatter_geo(
    df.head(50),
    lat="lat",
    lon="lng",
    size="population",
    hover_name="city",
    projection="natural earth",
    title="World Cities"
)

fig.show()

top = df.nlargest(20, "population")

plt.figure(figsize=(10, 6))
squarify.plot(sizes=top["population"], label=top["city"])
plt.axis("off")
plt.title("Population Distribution of Cities")
plt.show()
```

Or Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

df = pd.read_csv("crop_yield_data.csv").dropna()

rain_col = next(c for c in df.columns if "rain" in c.lower())
```

```
yield_col = next(c for c in df.columns if "yield" in c.lower())

X = df[[rain_col]]
y = df[yield_col]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("R2 Score:", r2_score(y_test, pred))

plt.scatter(X_test, y_test)
plt.plot(X_test, pred)
plt.xlabel("Rainfall")
plt.ylabel("Crop Yield")
plt.title("Rainfall vs Crop Yield")
plt.show()
```

SLIP 19 -

Q1

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Fuel_Efficiency": [18, 20, 22, 21, 55],
    "Engine_Power": [90, 110, 130, 125, 300],
    "Vehicle_Weight": [1000, 1200, 1300, 1250, 1400],
    "Engine_Size": [1.2, 1.5, 1.8, 1.6, 2.0],
```

```
    "Acceleration": [12, 11, 10, 10.5, 9]
})

plt.boxplot(
    [df["Fuel_Efficiency"], df["Engine_Power"]],
    labels=["Fuel Efficiency", "Engine Power"]
)
plt.title("Vehicle Outlier Detection")
plt.show()

for col in ["Fuel_Efficiency", "Engine_Power"]:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    print(col)
    print(df[(df[col] < Q1 - 1.5 * IQR) |
             (df[col] > Q3 + 1.5 * IQR)])
```

Q2

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df = pd.DataFrame({
    "Temperature": [20, 25, 30, 35, 40],
    "Humidity": [40, 50, 60, 70, 80],
    "Wind Speed": [5, 8, 10, 12, 15],
    "Pressure": [1000, 1005, 1010, 1015, 1020],
    "Rainfall": [10, 20, 30, 40, 50]
})

minmax = pd.DataFrame(
    MinMaxScaler().fit_transform(df), columns=df.columns
)

standard = pd.DataFrame(
```

```
    StandardScaler().fit_transform(df), columns=df.columns
)

normalized = pd.DataFrame(
    Normalizer().fit_transform(df), columns=df.columns
)
print(minmax)
print(standard)
print(normalized)
```

Or Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

df = pd.read_csv("heart.csv").dropna()

for col in df.select_dtypes(include="object").columns:
    df[col] = LabelEncoder().fit_transform(df[col])

target = "target" if "target" in df.columns else df.columns[-1]

X = df.drop(columns=[target])
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

SLIP 20 -

Q1

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    "Age": [20, 25, 30, 35, 40],
    "Salary": [20000, 30000, 40000, 50000, 60000],
    "Experience": [1, 3, 5, 7, 10],
    "Working_Hours": [7, 8, 8, 9, 10],
    "Performance_Score": [60, 65, 72, 80, 90]
})

scaler = StandardScaler()
cols = df.columns

df[cols] = scaler.fit_transform(df[cols])

print(df)
```

Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib_venn import venn2
from wordcloud import WordCloud
```

```
df = pd.DataFrame({
    "Book Title": [
        "Python Basics",
        "Data Science",
        "AI Fundamentals",
        "Machine Learning",
        "Python Analytics",
        "AI for Data"
    ],
    "Author": ["A", "B", "C", "A", "D", "C"],
    "Publication Year": [2010, 2012, 2015, 2018, 2020, 2022],
    "Average Rating": [4.1, 4.3, 4.5, 4.6, 4.2, 4.4],
    "Number of Ratings": [100, 150, 180, 200, 170, 210]
})

set1 = set(df[df["Publication Year"] < 2018]["Author"])
set2 = set(df[df["Publication Year"] >= 2018]["Author"])

venn2([set1, set2], set_labels=("Earlier", "Later"))
plt.title("Authors by Publication Period")
plt.show()

text = " ".join(df["Book Title"])

wc = WordCloud(
    width=800,
    height=400,
    background_color="white"
).generate(text)

plt.imshow(wc, interpolation="bilinear")
plt.axis("off")
plt.show()
```

Or Q2

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Library", "Seminar"],
    ["Workshop", "Project"],
    ["Library", "Assignment"],
    ["Seminar", "Workshop"],
    ["Assignment", "Seminar"],
    ["Workshop", "Assignment"],
    ["Project", "Library"],
    ["Library", "Seminar", "Project"],
    ["Library", "Seminar", "Workshop"]
]

te = TransactionEncoder()

df = pd.DataFrame(
    te.fit_transform(transactions),
    columns=te.columns_
)

for support in [0.2, 0.3]:
    freq = apriori(df, min_support=support, use_colnames=True)
    print(freq)
    print(association_rules(freq, metric="confidence", min_threshold=0.5))
```

SLIP 22 -

Q1

```
import pandas as pd
from sklearn.preprocessing import Normalizer

df = pd.DataFrame({
    "City": ["Nashik", "Pune", "Mumbai", "Nagpur", "Aurangabad"],
    "House_Type": ["Flat", "Bungalow", "Flat", "Flat", "Bungalow"],
    "House_Area": [800, 1500, 1000, 900, 1400],
    "Number_of_Rooms": [2, 4, 3, 2, 4],
    "Monthly_Rent": [12000, 30000, 22000, 14000, 28000]
})

cols = ["House_Area", "Number_of_Rooms", "Monthly_Rent"]

df[cols] = Normalizer().fit_transform(df[cols])

print(df)
```

Q2

```
import matplotlib.pyplot as plt
from matplotlib_venn import venn2

netflix = {"A", "B", "C", "D", "E", "F"}
amazon = {"D", "E", "F", "G", "H"}

venn2(
    [netflix, amazon],
    set_labels=("Netflix", "Amazon Prime")
)

plt.title("Netflix and Amazon Prime Users")
plt.show()
```

```
values = [
    len(netflix - amazon),
    len(amazon - netflix),
    len(netflix & amazon)
]

labels = ["Only Netflix", "Only Amazon Prime", "Both"]

plt.pie(values, labels=labels, autopct="%1.1f%%")
plt.title("Customer Distribution")
plt.show()
```

Or Q2

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

df = pd.DataFrame({
    "Attendance": [60, 65, 70, 75, 80, 85, 90, 95, 55, 68],
    "Marks": [45, 50, 55, 60, 65, 72, 80, 88, 40, 58]
})

scaler = StandardScaler()
X = scaler.fit_transform(df)

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(X)

plt.scatter(df["Attendance"], df["Marks"], c=labels, cmap="viridis")

centers = scaler.inverse_transform(model.cluster_centers_)

plt.scatter(
    centers[:, 0], centers[:, 1],
```

```
    marker="X", s=200
)
plt.xlabel("Attendance")
plt.ylabel("Marks")
plt.title("Student Clusters")
plt.show()
```

SLIP 23 -

Q1

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage

df = pd.DataFrame({
    "Product_name": ["P1", "P2", "P3", "P4", "P5"],
    "Price": [500, 700, 450, 900, 650],
    "Rating": [4.2, 4.5, 3.9, 4.8, 4.3],
    "Number_of_Sales": [120, 180, 90, 250, 160]
})

Z = linkage(
    df[["Price", "Rating", "Number_of_Sales"]],
    method="ward"
)

dendrogram(Z)

plt.title("Product Dendrogram")
plt.xlabel("Products")
plt.ylabel("Distance")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler, Normalizer

df = pd.DataFrame({
    "Temperature": [20, 25, 30, 35, 40],
    "Humidity": [40, 50, 60, 70, 80],
    "Wind Speed": [5, 8, 10, 12, 15],
    "Pressure": [1000, 1005, 1010, 1015, 1020],
    "Rainfall": [10, 20, 30, 40, 50]
})

print(pd.DataFrame(
    MinMaxScaler().fit_transform(df), columns=df.columns
))

print(pd.DataFrame(
    StandardScaler().fit_transform(df), columns=df.columns
))

print(pd.DataFrame(
    Normalizer().fit_transform(df), columns=df.columns
))
```

Or Q2

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

df = pd.read_csv("Students_Grading_Dataset.csv").dropna()

for col in df.select_dtypes(include="object").columns:
```

```
    df[col] = LabelEncoder().fit_transform(df[col])

target = next(
    (c for c in ["Performance", "Performance_Category", "Category", "Grade",
"Result"]
     if c in df.columns),
    df.columns[-1]
)

X = df.drop(columns=[target]).select_dtypes(include="number")
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted"))
print("Recall:", recall_score(y_test, pred, average="weighted"))
print("F1 Score:", f1_score(y_test, pred, average="weighted"))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

SLIP 25 -

Q1

```
import matplotlib.pyplot as plt

courses = ["BCA", "BBA", "B.Com", "B.Sc", "BA"]
students = [60, 45, 75, 50, 35]

plt.bar(
    courses, students,
    color=["red", "blue", "green", "orange", "purple"]
)

plt.xlabel("Courses")
plt.ylabel("Number of Students")
plt.title("Students Enrolled in Courses")
plt.show()
```

Q2

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df1 = pd.DataFrame({
    "Brand": ["Apple", "Samsung", "OnePlus", "Vivo"],
    "Model": ["A1", "S1", "O1", "V1"],
    "Price": [70000, 50000, 35000, 25000],
    "RAM": [8, 8, 12, 8]
})

df2 = pd.DataFrame({
    "Model": ["A1", "S1", "O1", "V1"],
    "Storage": [128, 256, 256, 128],
    "Battery_Capacity": [4000, 5000, 4500, 5000],
    "Operating_System": ["iOS", "Android", "Android", "Android"]
```

```
})

merged = pd.merge(df1, df2, on="Model")

le = LabelEncoder()
merged["Brand_Encoded"] = le.fit_transform(merged["Brand"])

merged = pd.get_dummies(
    merged, columns=["Operating_System"], dtype=int
)

print(df1)
print(df2)
print(merged)
```

Or Q2

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

transactions = [
    ["Basmati Rice", "Toor Dal", "Turmeric Powder", "Cooking Oil"],
    ["Atta", "Sugar", "Tea", "Milk"],
    ["Rice", "Moong Dal", "Salt", "Cooking Oil"],
    ["Wheat Flour", "Chickpeas", "Turmeric Powder", "Cumin Seeds"],
    ["Poha", "Peanuts", "Onions", "Cooking Oil"],
    ["Basmati Rice", "Paneer", "Garam Masala", "Tomatoes"],
    ["Atta", "Potatoes", "Onions", "Cooking Oil"]
]

te = TransactionEncoder()
df = pd.DataFrame(
    te.fit_transform(transactions),
    columns=te.columns_
)
```

```
for support in [0.3, 0.4]:
    freq = apriori(df, min_support=support, use_colnames=True)
    print(freq)
    print(association_rules(freq, metric="confidence", min_threshold=0.5))
```